

Recommender Systems

Per-Item Features, Collaborative Filtering, Binary Labels, and Mean Normalization

Rokan Uddin Faruqui
Professor, Department of CSE

University of Chittagong
rufaruqui@cu.ac.bd

Week 8 · Part 1

The most commercially important ML application you use daily

Where recommenders decide what you see

- Online shops: "customers who bought this also bought".
- Video and music streaming: the next episode, the next song.
- News feeds and social media: which posts appear first.
- Food delivery and ride apps: restaurants, routes, offers.

For many large platforms a large share of sales or viewing time comes from recommendations — and a one-percent improvement is worth a great deal.

Today

1. Making recommendations: the ratings matrix
2. Using per-item features (content-based)
3. Collaborative filtering — watched as it learns
4. Binary labels: likes, favourites, clicks
5. Mean normalization and the new user
6. TensorFlow implementation

Part 2: related items, deep content-based filtering, large catalogues, ethics, and principal component analysis.

Making Recommendations

The ratings matrix

	Alice	Bob	Carol	Dave
Curry	5	5	0	0
Noodles	5	?	?	0
Soup	?	4	0	?
Ginger Cake	0	0	5	4
Ice Cream	0	0	5	?

ratings 0–5 stars; ? = not rated

Notation

n_u users, n_m items (here $n_u = 4$, $n_m = 5$).
$r(i, j) = 1$ if user j has rated dish i.
$y^{(i,j)}$ = the rating, when $r(i, j) = 1$.

The task

Predict the ? entries, then recommend each user the unrated items with the highest predicted rating.

Key idea

Even by eye: Alice and Bob love spicy food and dislike sweets; Carol and Dave are the opposite. Would Alice like *Pepper Soup*? Probably.

Real matrices are huge and almost empty.

Using Per-Item Features

If we know what each dish is like . . .

dish	x_1 spicy	x_2 sweet
Chicken Curry	0.90	0
Chilli Noodles	1.00	0.01
Pepper Soup	0.99	0
Ginger Cake	0.10	1.00
Mango Ice Cream	0	0.90

A linear model per user

Predict user j's rating of dish i as

$$\mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)} + b^{(j)}.$$

Each user has their own $\mathbf{w}^{(j)}, b^{(j)}$: *linear regression, once per user*, on the dishes that user rated.

Worked example

Suppose Alice's parameters are $\mathbf{w}^{(1)} = (5,\ 0)$, $b^{(1)} = 0$. For *Pepper Soup*, $\mathbf{x}^{(3)} = (0.99,\ 0)$:

$$\mathbf{w}^{(1)} \cdot \mathbf{x}^{(3)} + b^{(1)} = 5(0.99) + 0(0) + 0 = \mathbf{4.95}$$

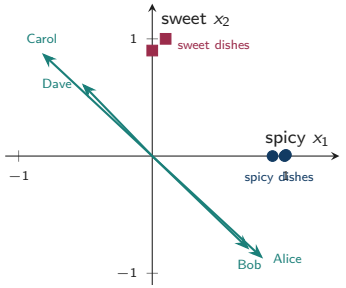
— recommend it.

Cost for user j

$$\frac{1}{2} \sum_{i:\, r(i,j)=1} \left(\mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)} + b^{(j)} - y^{(i,j)}\right)^2 + \frac{\lambda}{2} \sum_{k} \left(w_k^{(j)}\right)^2$$

Sum over users to learn all $\mathbf{w}^{(j)}, b^{(j)}$ at once.

Picture it: users are directions, dishes are points



arrows: each user's $\mathbf{w}^{(j)}$ (half length), fitted with $\lambda = 1$

The dot product is agreement

$\mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)}$ is large when the dish lies in the direction of the user's arrow: Alice points to spicy (blue), Carol to sweet (rose).

Fitted with $\lambda = 1$

Alice: $\mathbf{w} = (1.66,\ -1.75)$, $b = 2.51$; predicted rating of *Pepper Soup* $= 4.15$.

Dave: $\mathbf{w} = (-1.05,\ 1.24)$; predicted rating of *Mango Ice Cream* $= 2.74$.

Where do the features come from?

Someone had to score every dish by hand — impossible for millions of items. Next: learn the features too.

Collaborative Filtering

Turn the problem around

If we know the users . . .

Suppose we knew every user's $\mathbf{w}^{(j)}, b^{(j)}$. Then for each dish i we could *learn* its features $\mathbf{x}^{(i)}$ from the ratings it received:

$$\min_{\mathbf{x}^{(i)}} \frac{1}{2} \sum_{j:\, r(i,j)=1} \left(\mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)} + b^{(j)} - y^{(i,j)}\right)^2 + \frac{\lambda}{2} \sum_{k} \left(x_k^{(i)}\right)^2$$

Possible only because *many* users rated each dish: their ratings collectively say what the dish is like.

Collaborative filtering cost

Learn $\mathbf{w}$, b *and* $\mathbf{x}$ together:

$$J = \frac{1}{2} \sum_{(i,j):\, r(i,j)=1} \left(\mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)} + b^{(j)} - y^{(i,j)}\right)^2 + \frac{\lambda}{2} \sum_{j,k} \left(w_k^{(j)}\right)^2 + \frac{\lambda}{2} \sum_{i,k} \left(x_k^{(i)}\right)^2$$

Gradient descent on all three:

$$w_k^{(j)} \mathrel{-}= \alpha \frac{\partial J}{\partial w_k^{(j)}}, \quad b^{(j)} \mathrel{-}= \alpha \frac{\partial J}{\partial b^{(j)}}, \quad x_k^{(i)} \mathrel{-}= \alpha \frac{\partial J}{\partial x_k^{(i)}}$$

"Collaborative": every user's ratings help learn every dish's features, which help predict every other user's ratings.

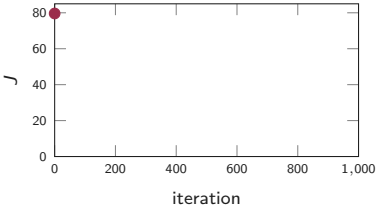
Watch collaborative filtering fill in the matrix

	Alice	Bob	Carol	Dave
Curry	5	5	0	0
Noodles	5	-0.01	-0.23	0
Soup	0.07	4	0	-0.02
Ginger Cake	0	0	5	4
Ice Cream	0	0	5	-0.02

blue: observed ratings (fixed) dashed: predictions for the ? entries

Training

iteration $t = 0$: $J = 79.614$



Two learned features per dish and user, $\lambda = 0.1$, $\alpha = 0.01$, random start. No taste labels were given.

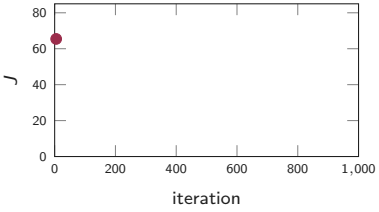
Watch collaborative filtering fill in the matrix

	Alice	Bob	Carol	Dave
Curry	5	5	0	0
Noodles	5	0.34	0.16	0
Soup	0.54	4	0	0.16
Ginger Cake	0	0	5	4
Ice Cream	0	0	5	0.17

blue: observed ratings (fixed) dashed: predictions for the ? entries

Training

iteration $t = 5$: $J = 65.451$



Two learned features per dish and user, $\lambda = 0.1$, $\alpha = 0.01$, random start. No taste labels were given.

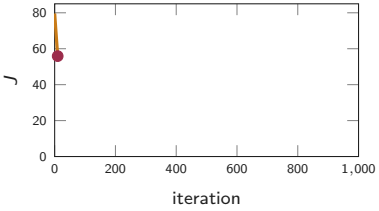
Watch collaborative filtering fill in the matrix

	Alice	Bob	Carol	Dave
Curry	5	5	0	0
Noodles	5	0.63	0.46	0
Soup	0.92	4	0	0.31
Ginger Cake	0	0	5	4
Ice Cream	0	0	5	0.34

blue: observed ratings (fixed) dashed: predictions for the ? entries

Training

iteration $t = 10$: $J = 55.896$



Two learned features per dish and user, $\lambda = 0.1$, $\alpha = 0.01$, random start. No taste labels were given.

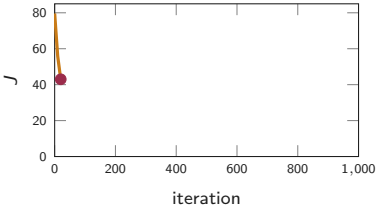
Watch collaborative filtering fill in the matrix

	Alice	Bob	Carol	Dave
Curry	5	5	0	0
Noodles	5	1.07	0.77	0
Soup	1.54	4	0	0.53
Ginger Cake	0	0	5	4
Ice Cream	0	0	5	0.71

blue: observed ratings (fixed) dashed: predictions for the ? entries

Training

iteration $t = 20$: $J = 42.985$



Two learned features per dish and user, $\lambda = 0.1$, $\alpha = 0.01$, random start. No taste labels were given.

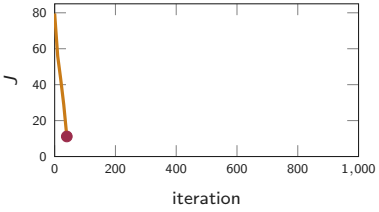
Watch collaborative filtering fill in the matrix

	Alice	Bob	Carol	Dave
Curry	5	5	0	0
Noodles	5	2.20	0.12	0
Soup	2.94	4	0	0.38
Ginger Cake	0	0	5	4
Ice Cream	0	0	5	1.99

blue: observed ratings (fixed) dashed: predictions for the ? entries

Training

iteration $t = 40$: $J = 11.173$



Two learned features per dish and user, $\lambda = 0.1$, $\alpha = 0.01$, random start. No taste labels were given.

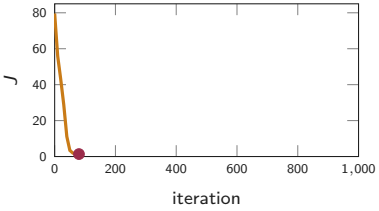
Watch collaborative filtering fill in the matrix

	Alice	Bob	Carol	Dave
Curry	5	5	0	0
Noodles	5	3.65	-0.64	0
Soup	4.45	4	0	0.18
Ginger Cake	0	0	5	4
Ice Cream	0	0	5	3.47

blue: observed ratings (fixed) dashed: predictions for the ? entries

Training

iteration $t = 80$: $J = 1.369$



Two learned features per dish and user, $\lambda = 0.1$, $\alpha = 0.01$, random start. No taste labels were given.

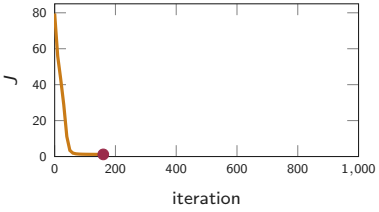
Watch collaborative filtering fill in the matrix

	Alice	Bob	Carol	Dave
Curry	5	5	0	0
Noodles	5	3.86	-0.56	0
Soup	4.57	4	0	0.27
Ginger Cake	0	0	5	4
Ice Cream	0	0	5	3.85

blue: observed ratings (fixed) dashed: predictions for the ? entries

Training

iteration $t = 160$: $J = 1.187$



Two learned features per dish and user, $\lambda = 0.1$, $\alpha = 0.01$, random start. No taste labels were given.

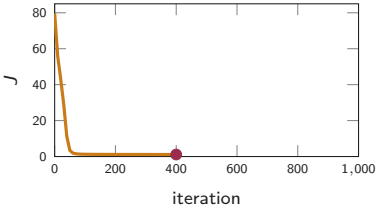
Watch collaborative filtering fill in the matrix

	Alice	Bob	Carol	Dave
Curry	5	5	0	0
Noodles	5	4.04	-0.41	0
Soup	4.61	4	0	0.27
Ginger Cake	0	0	5	4
Ice Cream	0	0	5	3.93

blue: observed ratings (fixed) dashed: predictions for the ? entries

Training

iteration $t = 400$: $J = 1.123$



Two learned features per dish and user, $\lambda = 0.1$, $\alpha = 0.01$, random start. No taste labels were given.

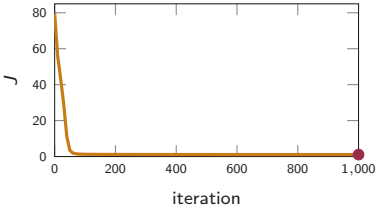
Watch collaborative filtering fill in the matrix

	Alice	Bob	Carol	Dave
Curry	5	5	0	0
Noodles	5	4.27	-0.25	0
Soup	4.65	4	0	0.27
Ginger Cake	0	0	5	4
Ice Cream	0	0	5	3.94

blue: observed ratings (fixed) dashed: predictions for the ? entries

Training

iteration $t = 1000$: $J = 1.080$



Two learned features per dish and user, $\lambda = 0.1$, $\alpha = 0.01$, random start. No taste labels were given.

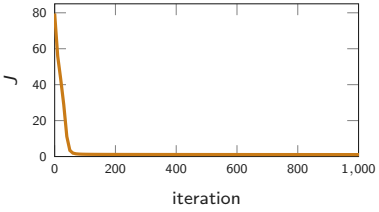
Watch collaborative filtering fill in the matrix

	Alice	Bob	Carol	Dave
Curry	5	5	0	0
Noodles	5	4.47	-0.19	0
Soup	4.62	4	0	0.30
Ginger Cake	0	0	5	4
Ice Cream	0	0	5	3.94

blue: observed ratings (fixed) dashed: predictions for the ? entries

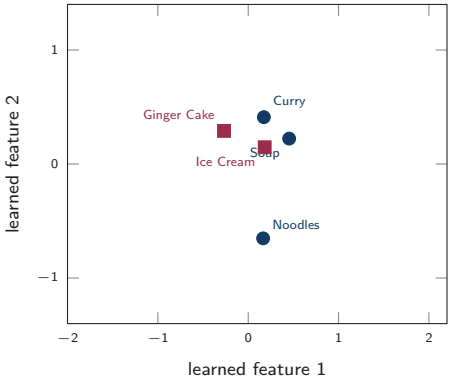
Training

iteration $t = 2000$: $J = 1.060$



Two learned features per dish and user, $\lambda = 0.1$, $\alpha = 0.01$, random start. No taste labels were given.

The same run, seen in feature space



$t = 0$

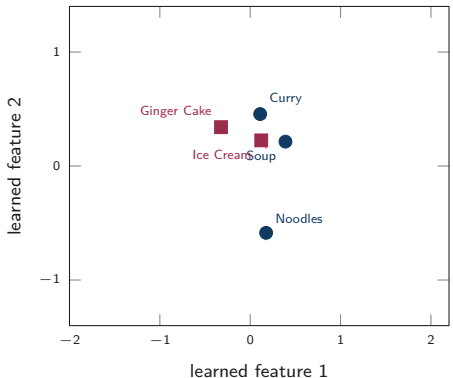
Features nobody named

The dishes start at random points. As training proceeds the three spicy dishes drift to one side and the two sweet dishes to the other — separated by the learned features, although the algorithm was never told what spicy or sweet is. It discovered the distinction from who liked what.

Learned features have no fixed meaning

The axes are not “spicy” and “sweet”. Any rotation of the feature space, with the users rotated to match, gives the same predictions. Only distances and dot products mean anything.

The same run, seen in feature space



$t = 5$

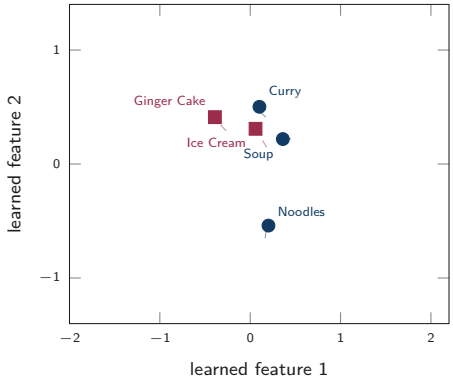
Features nobody named

The dishes start at random points. As training proceeds the three spicy dishes drift to one side and the two sweet dishes to the other — separated by the learned features, although the algorithm was never told what spicy or sweet is. It discovered the distinction from who liked what.

Learned features have no fixed meaning

The axes are not “spicy” and “sweet”. Any rotation of the feature space, with the users rotated to match, gives the same predictions. Only distances and dot products mean anything.

The same run, seen in feature space



$t = 10$

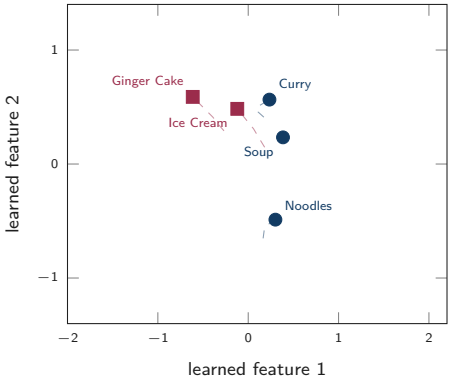
Features nobody named

The dishes start at random points. As training proceeds the three spicy dishes drift to one side and the two sweet dishes to the other — separated by the learned features, although the algorithm was never told what spicy or sweet is. It discovered the distinction from who liked what.

Learned features have no fixed meaning

The axes are not “spicy” and “sweet”. Any rotation of the feature space, with the users rotated to match, gives the same predictions. Only distances and dot products mean anything.

The same run, seen in feature space



$t = 20$

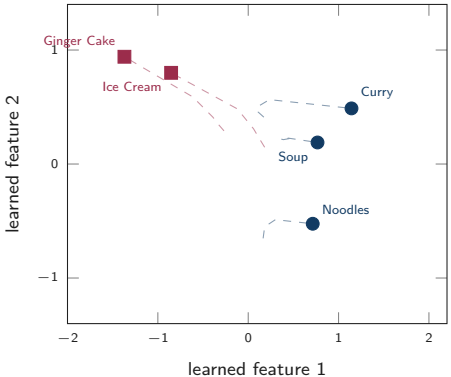
Features nobody named

The dishes start at random points. As training proceeds the three spicy dishes drift to one side and the two sweet dishes to the other — separated by the learned features, although the algorithm was never told what spicy or sweet is. It discovered the distinction from who liked what.

Learned features have no fixed meaning

The axes are not “spicy” and “sweet”. Any rotation of the feature space, with the users rotated to match, gives the same predictions. Only distances and dot products mean anything.

The same run, seen in feature space



$t = 40$

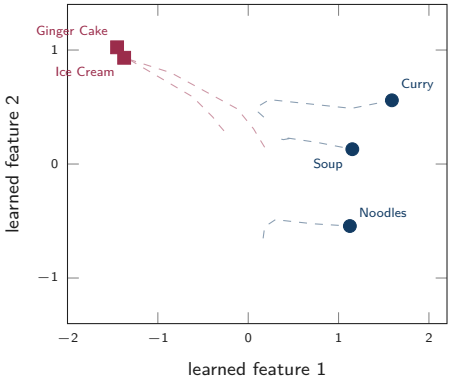
Features nobody named

The dishes start at random points. As training proceeds the three spicy dishes drift to one side and the two sweet dishes to the other — separated by the learned features, although the algorithm was never told what spicy or sweet is. It discovered the distinction from who liked what.

Learned features have no fixed meaning

The axes are not “spicy” and “sweet”. Any rotation of the feature space, with the users rotated to match, gives the same predictions. Only distances and dot products mean anything.

The same run, seen in feature space



$t = 80$

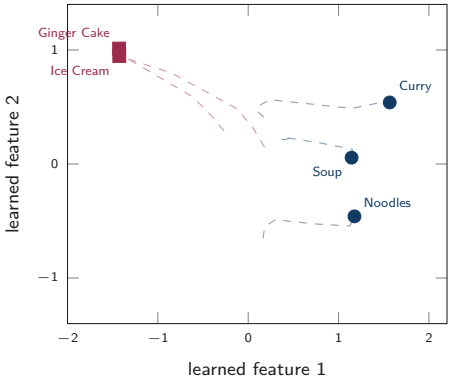
Features nobody named

The dishes start at random points. As training proceeds the three spicy dishes drift to one side and the two sweet dishes to the other — separated by the learned features, although the algorithm was never told what spicy or sweet is. It discovered the distinction from who liked what.

Learned features have no fixed meaning

The axes are not “spicy” and “sweet”. Any rotation of the feature space, with the users rotated to match, gives the same predictions. Only distances and dot products mean anything.

The same run, seen in feature space



$t = 160$

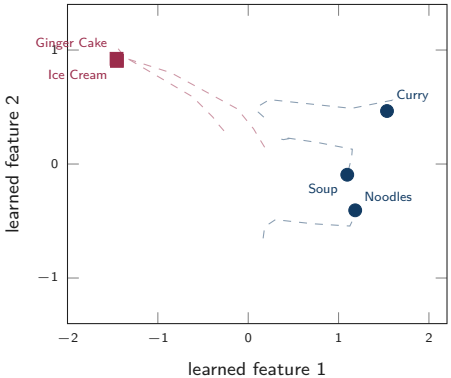
Features nobody named

The dishes start at random points. As training proceeds the three spicy dishes drift to one side and the two sweet dishes to the other — separated by the learned features, although the algorithm was never told what spicy or sweet is. It discovered the distinction from who liked what.

Learned features have no fixed meaning

The axes are not “spicy” and “sweet”. Any rotation of the feature space, with the users rotated to match, gives the same predictions. Only distances and dot products mean anything.

The same run, seen in feature space



$t = 400$

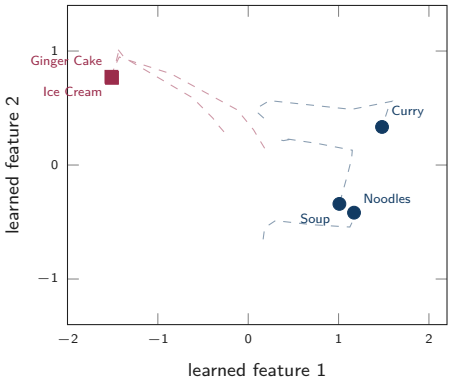
Features nobody named

The dishes start at random points. As training proceeds the three spicy dishes drift to one side and the two sweet dishes to the other — separated by the learned features, although the algorithm was never told what spicy or sweet is. It discovered the distinction from who liked what.

Learned features have no fixed meaning

The axes are not “spicy” and “sweet”. Any rotation of the feature space, with the users rotated to match, gives the same predictions. Only distances and dot products mean anything.

The same run, seen in feature space



$t = 1000$

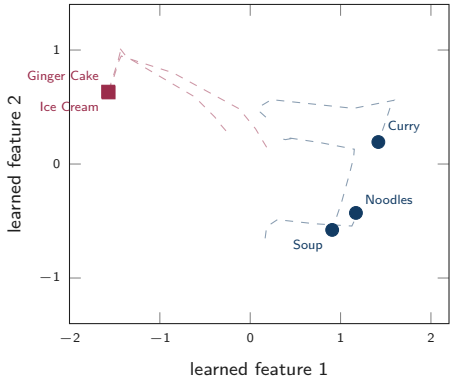
Features nobody named

The dishes start at random points. As training proceeds the three spicy dishes drift to one side and the two sweet dishes to the other — separated by the learned features, although the algorithm was never told what spicy or sweet is. It discovered the distinction from who liked what.

Learned features have no fixed meaning

The axes are not “spicy” and “sweet”. Any rotation of the feature space, with the users rotated to match, gives the same predictions. Only distances and dot products mean anything.

The same run, seen in feature space



$t = 2000$

Features nobody named

The dishes start at random points. As training proceeds the three spicy dishes drift to one side and the two sweet dishes to the other — separated by the learned features, although the algorithm was never told what spicy or sweet is. It discovered the distinction from who liked what.

Learned features have no fixed meaning

The axes are not “spicy” and “sweet”. Any rotation of the feature space, with the users rotated to match, gives the same predictions. Only distances and dot products mean anything.

What the trained model predicts

	Alice	Bob	Carol	Dave
Curry	5	5	0	0
Noodles	5	4.47	-0.19	0
Soup	4.62	4	0	0.30
Ginger Cake	0	0	5	4
Ice Cream	0	0	5	3.94

The five missing ratings

Bob, *Chilli Noodles*	4.47
Carol, *Chilli Noodles*	−0.19
Alice, *Pepper Soup*	4.62
Dave, *Pepper Soup*	0.30
Dave, *Mango Ice Cream*	3.94

Key idea

Recommend *Pepper Soup* to Alice and *Mango Ice Cream* to Dave. Observed ratings are reproduced closely (e.g. 4.99 for Alice's 5): the small regularization keeps it from being exact.

Final cost $J = 1.060$ after 2,000 iterations.

Binary Labels: Likes, Favourites and Clicks

Many systems never ask for a rating

Implicit, binary signals

- Did the user buy the item after seeing it? (1 / 0)
- Did they like, favourite or share the post?
- Did they order the same dish again within a month?
- Did they click the item when shown?

1 = engaged, 0 = shown but not engaged, ? = never shown.

From regression to classification

Exactly the step from Week 1 to Week 3:

$$P\left(y^{(i,j)}=1\right)=g\left(\mathbf{w}^{(j)}\cdot\mathbf{x}^{(i)}+b^{(j)}\right),\quad g(z)=\tfrac{1}{1+e^{-z}}$$

Binary collaborative filtering cost

$$J=\sum_{(i,j):\,r(i,j)=1}L\left(f^{(i,j)},\,y^{(i,j)}\right)+\tfrac{\lambda}{2}\|\mathbf{W}\|^2+\tfrac{\lambda}{2}\|\mathbf{X}\|^2$$

with the binary cross-entropy $L(f,y)=-y\ln f-(1-y)\ln(1-f)$.

A "0" is not a dislike

"Shown but not clicked" may mean the user never noticed it. And items never shown are *missing*, not 0: treating them as dislikes teaches the model that whatever it did not recommend is bad.

Mean Normalization

A new user, Eve, has rated nothing

Without normalization

Eve's terms appear only in the penalty $\frac{\lambda}{2}\|\mathbf{w}^{(5)}\|^2$, so gradient descent drives $\mathbf{w}^{(5)} \to \mathbf{0}$ (here $(-0.007,\ -0.002)$), and $b^{(5)}$ never moves from 0. Every prediction for Eve is ≈ 0:

$$(-0.01,\ -0.01,\ -0.00,\ 0.01,\ 0.01)$$

The system thinks Eve hates everything — the worst possible start.

With normalization

Subtract each dish's mean rating μ_i (over users who rated it) before training; add it back when predicting:

$$\text{prediction} = \mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)} + b^{(j)} + \mu_i.$$

dish	μ_i	Eve, raw	Eve, normalized
Curry	2.50	−0.01	2.49
Noodles	2.50	−0.01	2.49
Soup	2.00	−0.00	1.99
Ginger Cake	2.25	0.01	2.26
Ice Cream	1.67	0.01	1.68

Key idea

With normalization a new user is predicted the *average* rating of each dish — a sensible default until their own ratings arrive. It also helps training converge faster.

TensorFlow Implementation

Automatic differentiation does the calculus

```
X = tf.Variable(rng.normal(0, .5, (n_m, n)))
W = tf.Variable(rng.normal(0, .5, (n_u, n)))
b = tf.Variable(tf.zeros((1, n_u)))
opt = keras.optimizers.Adam(learning_rate=0.1)

for it in range(200):
    with tf.GradientTape() as tape:
        P = tf.matmul(X, W, transpose_b=True) + b
        E = (P - Ynorm) * R          # only rated
        J = 0.5 * tf.reduce_sum(E ** 2) \
            + lam / 2 * (tf.reduce_sum(X ** 2)
                         + tf.reduce_sum(W ** 2))
    grads = tape.gradient(J, [X, W, b])
    opt.apply_gradients(zip(grads, [X, W, b]))
```

Why a custom loop?

Collaborative filtering is not a `Sequential` stack of layers: the "inputs" $\mathbf{x}^{(i)}$ are themselves parameters. So we write the cost, and let `GradientTape` compute every partial derivative (Week 5's back-propagation, automated).

Vectorized

$\mathbf{XW}^{\mathsf{T}} + \mathbf{b}$ predicts the whole $n_m \times n_u$ matrix at once; multiplying by $\mathbf{R}$ zeroes the unrated entries so they contribute nothing.

Wrap-up and Assessment

Summary

What to remember

- The ratings matrix: n_m items $\times$ n_u users, mostly missing.
- Content-based: known item features, one linear regression per user.
- Collaborative filtering learns item features and user parameters together from the ratings alone.
- Binary signals (likes, clicks) use the logistic version.
- Mean normalization gives new users the average prediction.
- Implementation: write the cost, let `GradientTape` differentiate.

Formulas

$$\hat{y}^{(i,j)} = \mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)} + b^{(j)} + \mu_i$$

$$J = \frac{1}{2} \sum_{r(i,j)=1} (\hat{y}^{(i,j)} - y^{(i,j)})^2 + \frac{\lambda}{2}(\|\mathbf{W}\|^2 + \|\mathbf{X}\|^2)$$

Next

Part 2: finding related items, deep content-based filtering, recommending from millions of items, ethics, and PCA.

Descriptive questions

1. Define n_u, n_m, $r(i,j)$ and $y^{(i,j)}$, and state the goal of a recommender system in terms of the ratings matrix. **[4]**
2. Explain content-based prediction with per-item features. With $\mathbf{w} = (5, 0)$, $b = 0$, compute the predicted rating of a dish with $\mathbf{x} = (0.2,\ 0.9)$. **[5]**
3. Write the collaborative-filtering cost and explain why it contains penalty terms for both $\mathbf{w}$ and $\mathbf{x}$. **[6]**
4. Why is the method called *collaborative*? Why could it not learn a dish's features if each dish were rated by only one user? **[5]**
5. Explain why learned features have no fixed meaning, using the idea of rotating the feature space. **[5]**
6. From the trained model, which unrated dish should be recommended to each of Alice and Dave, and why? **[4]**

Descriptive questions (continued)

7. Give three sources of binary feedback, and write the binary collaborative-filtering cost. **[5]**
8. Why should items never shown to a user be treated as missing rather than as 0? **[4]**
9. Show that without mean normalization a user with no ratings receives predictions near 0. What do they receive with it? **[6]**
10. Compute the dish means μ_i for the lecture's matrix. **[4]**
11. Why is collaborative filtering implemented with a custom training loop rather than a `Sequential` model? **[4]**
12. Derive $\partial J / \partial x_k^{(i)}$ for the collaborative-filtering cost. **[8]**

Design questions

1. **Design a book recommender.** A university library wants to recommend books to students from five years of borrowing records (no ratings). Design the labels, how you treat books never borrowed, the cost function, and how you will evaluate it before launch. **[12]**
2. **Design for cold start.** A new food-delivery app has thousands of dishes but almost no users. Design how you combine per-item features and collaborative filtering as data accumulates. **[10]**
3. **Design an evaluation.** Design a held-out evaluation for a collaborative filtering model: which ratings you hide, the metric, and why a random split of ratings can be misleading for a recommender. **[12]**
4. **Design the number of features.** Design an experiment to choose n (the number of learned features) and λ, and say what overfitting looks like in this setting. **[10]**

Reading

Primary text

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

→ **§12.3** — missing values and matrix completion, with the dish-ratings (Netflix) example: the same low-rank model as collaborative filtering.

Supplementary

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

→ **§12.1–12.2** — linear latent-variable models; collaborative filtering is a latent-factor model of this family.

Lab (CSE 816, Week 8): collaborative filtering recommender system.